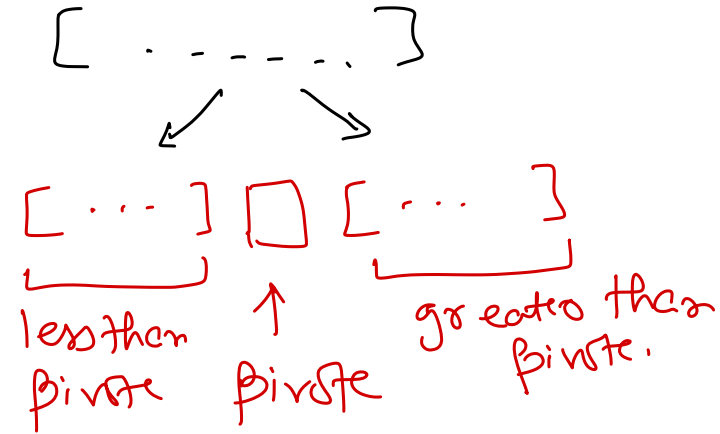


Quick Sort $\Rightarrow$ Divide and Conquer



Partitioning

Elements less than pivot are placed to left of pivot and elements greater than pivot are placed to right of pivot.

Sorting in ascending order.

[5 1 9 4 3 2 8 7]

↑
Pivot

↑
left ⇒ should point to element less than pivot

should point to element greater than pivot ← right

[5 1 9 4 3 2 8 7]

↑
Pivot

↑
left
↓
element is less than pivot ⇒ move to next element

↑
right

[5 1 9 4 3 2 8 7]

↑
Pivot

↑
left

↑
right

element is not
less than pivot
⇒ find an element
right should not
point to

[5 1 9 4 3 2 8 7]

↑
Pivot

↑
left

↑
right

element is
greater than pivot
⇒ move to previous
element

[5 1 9 4 3 2 8 7]

↑
Pivot

↑
left

↑
right

[5 1 ~~9~~² 4 3 ~~2~~⁹ 8 7]

↑
Pivot

↑
left

↑
right

element is not less than pivot

element is not greater than pivot

Swap the two elements.

[5 1 2 4 3 9 8 7]

↑
Pivot

↑
left

↑
right

element is less
than pivot $\Rightarrow$ move
to next element.

[5 1 2 4 3 9 8 7]

↑
Pivot

↑
left

↑
right

element is less
than pivot $\Rightarrow$ move
to next element.

[5 1 2 4 3 9 8 7]

↑
Pivot

↑ ↑
left right
↓

element is less
than pivot $\Rightarrow$ move
to next element.

[5 1 2 4 3 9 8 7]

↑
Pivot

↑ ↑
left right
↖

element is not
less than pivot $\Leftarrow$
↓
time to move right pointer

[5 1 2 4 3 9 8 7]

↑
Pivot

left ↑ right

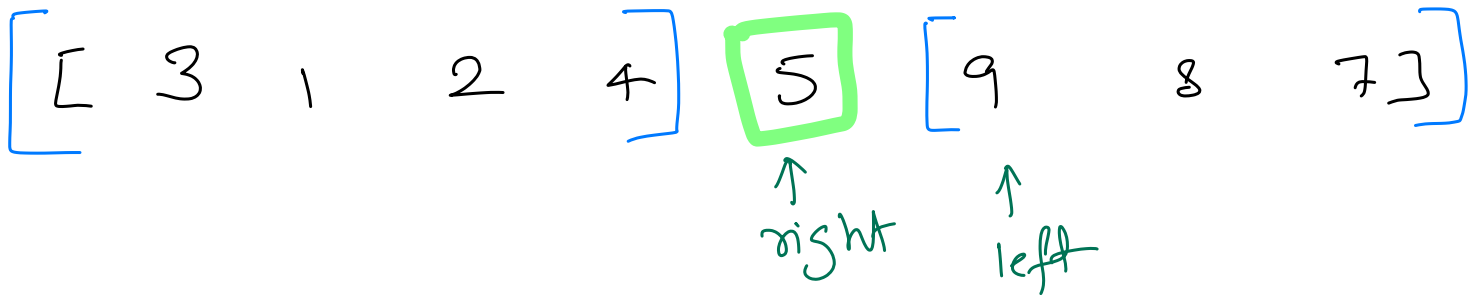
↘ element is greater than pivot
move to previous element

3
[~~5~~ 1 2 4 ~~3~~ 9 8 7]

↑
Pivot

↑ right left ↑

left and right has crossed over ⇒ STOP.
→ Swap pivot and right elements.



int Partition(arr, startPos, endPos)

- Set pivotPos to startPos
- Set left to startPos + 1
- Set right to endPos
- while (left <= right)
 - while ((left <= right) and (arr[left] <= arr[pivotPos]))
 - Increment left by 1
 - while ((left <= right) and (arr[pivotPos] < arr[right]))
 - Decrement right by 1
 - if (left < right)
 - Swap elements at left and right position
- Swap element at pivotPos and right
- Return right // After swap, pivot element at right

Partition

<u>Time</u>	<u>Space</u>
$O(n)$	$O(1)$

QuickSort(arr, startPos, endPos)

// base case = array sorted (number of elements ≤ 1).

- if (endPos - startPos + 1) ≤ 1
 - Stop

// Array too big - Partition them into two parts around pivot and sort each of them.

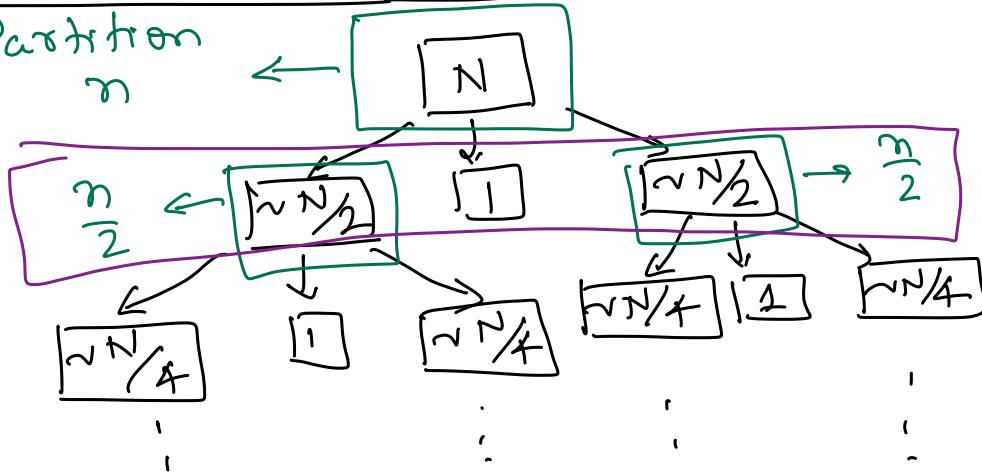
- pivotPos = Partition(arr, startPos, endPos)
- QuickSort(arr, startPos, pivotPos - 1)
- QuickSort(arr, pivotPos + 1, endPos)
- Stop

← Tail recursion.

Exercise: Remove tail recursion.

Time complexity of Quick Sort

Partition
 n



$$\frac{n}{2} + \frac{n}{2} = n$$

Steps?

$$\log_2 n$$

$\boxed{1} \quad \boxed{1} \quad \dots \quad \dots \quad \dots \quad \dots \quad \boxed{1} \quad]$

Time Complexity = number of steps $\times$ work done at each step

$$= \log_2 n \times n$$

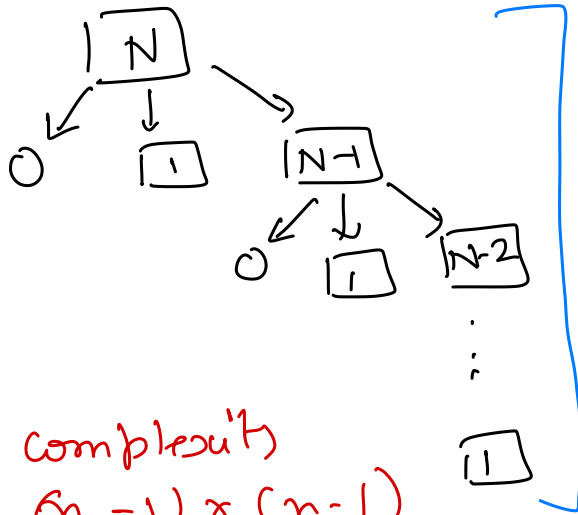
$$\Rightarrow O(n \log_2 n) \leftarrow \text{logarithmic}$$

Space complexity $\Rightarrow O(\log_2 n) \leftarrow$ On system stack to make recursive calls.

Best case and worse case for Quick Sort

[1 2 3 4 5]

As per our partitioning approach, worse case is when array is already sorted.



Steps?

$n-1$

Time complexity
 $= (n-1) \times (n-1)$
 $\approx n^2 \Rightarrow O(n^2)$

Space complexity $= O(n)$

Merge Sort → Divide and Conquer

- ① Divide input into smaller parts, until the smaller parts are sorted.
- ② Merge two sorted parts into one bigger sorted part.

[5 1 9 4 3 2 8 7]

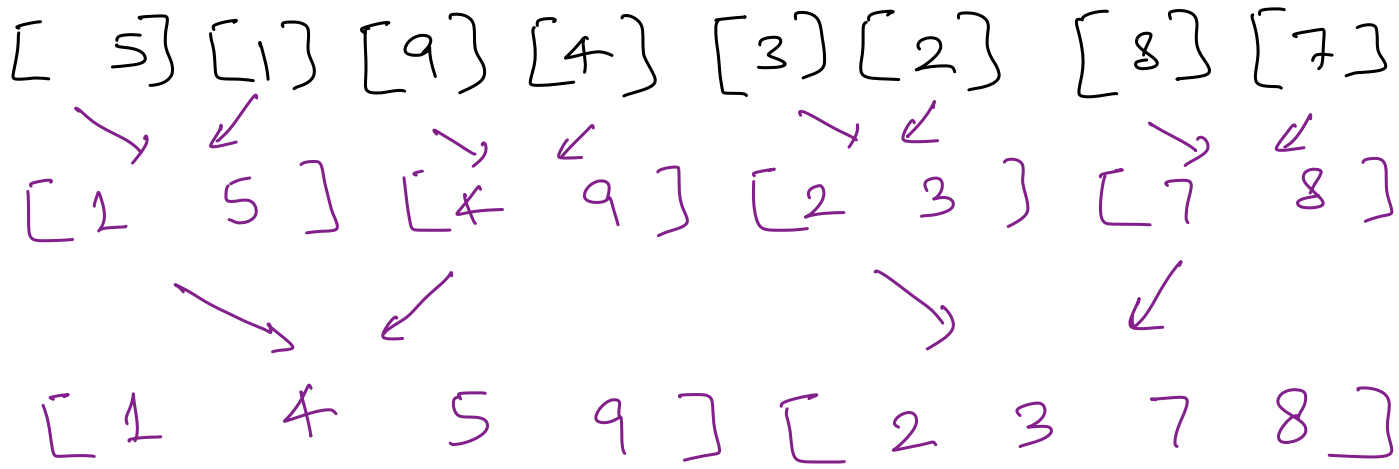


[5 1 9 4] [3 2 8 7]

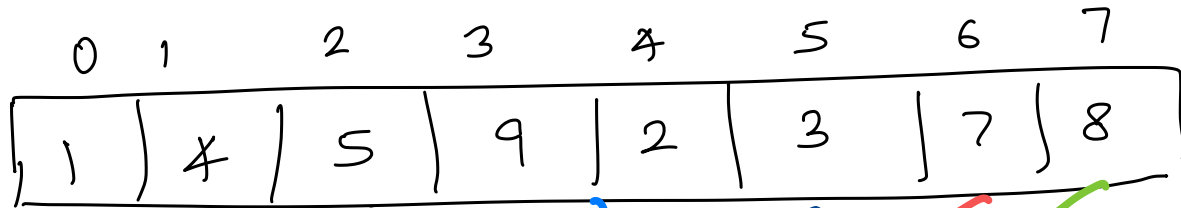


[5 1] [9 4] [3 2] [8 7]





- ① In place merge. $\Rightarrow$ Do not use extra space.
- ② Out of place merge. $\Rightarrow$ Uses $O(n)$ extra space.



Start Pos 1 $\rightarrow$ 0
End Pos 1 $\rightarrow$ 3

Start Pos 2 $\rightarrow$ 4
End Pos 2 $\rightarrow$ 7

0	1	2	3	4	5	6	7
1	2	3	4	5	7	8	9

merged array

$i_1 \rightarrow$ ~~0~~ ~~1~~ ~~2~~ 3

[start pos 1, end pos 1]

$i_2 \rightarrow$ ~~4~~ ~~5~~ ~~6~~ ~~7~~ 8

[start pos 2, end pos 2]

$r \rightarrow$ ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~
~~5~~ ~~6~~ 7

MergeSort(arr, startPos, endPos)

// base case = array sorted (number of elements ≤ 1).

- if (endPos - startPos + 1) ≤ 1
 - Stop

// Array too big - Divide them into two parts and sort each of them.

- middlePos = (startPos + endPos) / 2

- MergeSort(arr, startPos, middlePos)

- MergeSort(arr, middlePos + 1, endPos)

// We got two sorted arrays, merge them into one

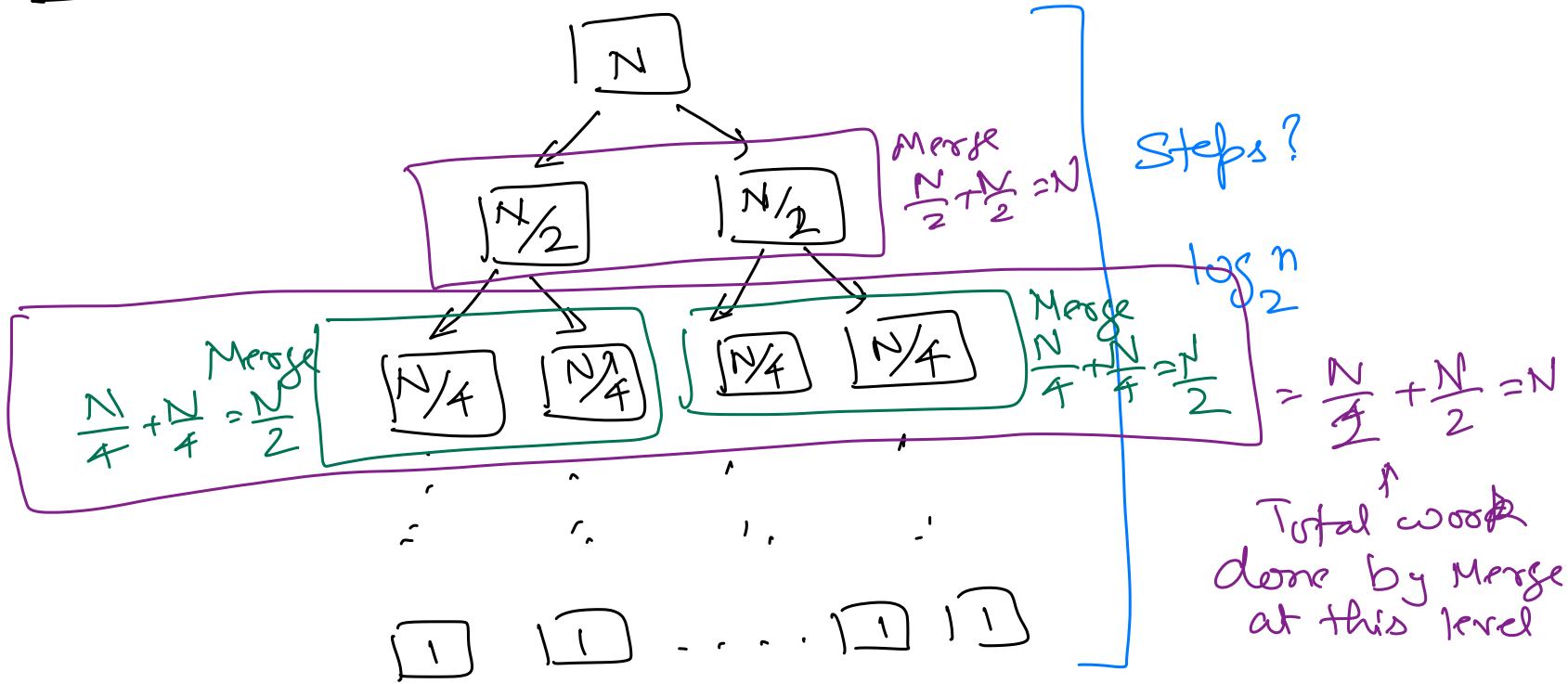
- Merge(arr, startPos, middlePos, middlePos + 1, endPos)

- Stop

Space complexity: space taken by merge
+ space on system stack due
to recursion

$$= n + \log_2 n \Rightarrow O(n)$$

Time Complexity of Merge Sort



Time Complexity = number of steps $\times$ work done in each step

$$= \log_2 n \times n \Rightarrow O(n \log_2 n)$$

Merge(arr, startPos1, endPos1, startPos2, endPos2)

- Set i1 to startPos1
- Set i2 to startPos2
- Set r to 0
- while ((i1 <= endPos1) and (i2 <= endPos2))

m n elements

- if (arr[i1] < arr[i2])
- mergedArray[r] = arr[i1]
- Increment i1 by 1

else

- mergedArray[r] = arr[i2]
- Increment i2 by 1
- Increment r by 1

(m or n times.)

m times
+ x

// Copy remaining elements from other sorted array into mergedArray

- while (i1 <= endPos1)
- mergedArray[r] = arr[i1]
- Increment i1 by 1
- Increment r by 1

0

- while (i2 <= endPos2)
- mergedArray[r] = arr[i2]
- Increment i2 by 1
- Increment r by 1

n-x times

// Copy merged elements back to arr

- Set i to startPos1
- Set j to 0
- while (i < r)
- arr[i] = mergedArray[j]
- Increment i by 1
- Increment j by 1
- Stop

m+n times.

Time Complexity

$$= (m+n) + (m+n)$$

$$= 2(m+n)$$

$$\Rightarrow O(m+n)$$

$$m+x + 0 + n-x$$

$$= m+n \text{ times.}$$

Space Complexity

$$O(m+n)$$

↑

Space used to
store merged
elements.

External Sorting → Data to be sorted is stored on secondary storage.

Internal Sorting → Data to be sorted is stored entirely in primary memory.

m-way merge sort → Divide into m parts.

Heap Sort

Sorting that uses Heap to sort data.

↓
Ascending
⇓
Max Heap

↓
Descending
⇓
Min Heap

↓
Is a binary tree in which
each node satisfies heap property.

Each parent
node value $>$
value of the
child nodes.

Max Heap

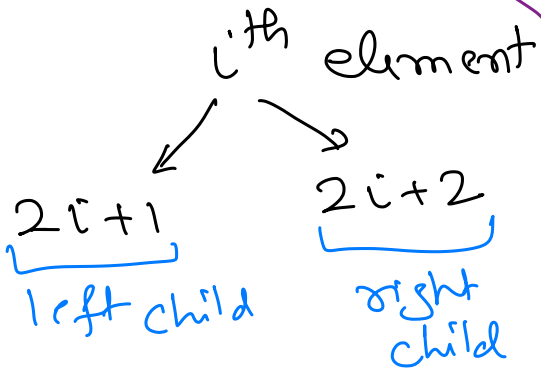
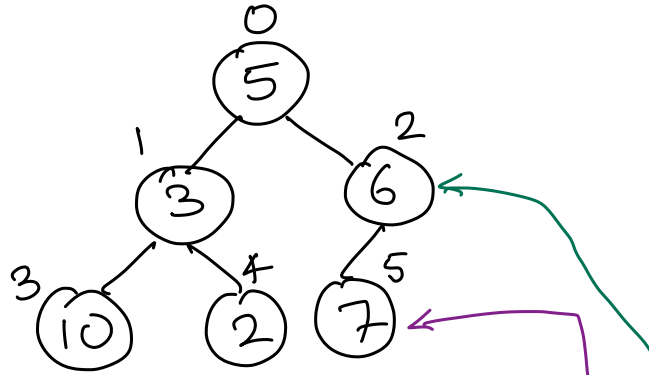
⇓
Largest element
is stored in
root node.

Each parent
node value
 $<$ value
of the
child
nodes.

Min Heap

⇓
Smallest
element is
stored in root.

0	1	2	3	4	5
5	3	6	10	2	7



Complete Binary Tree

① Fill one level with all possible nodes before moving to next level.

② when adding a new node at a level, its added from left to right.

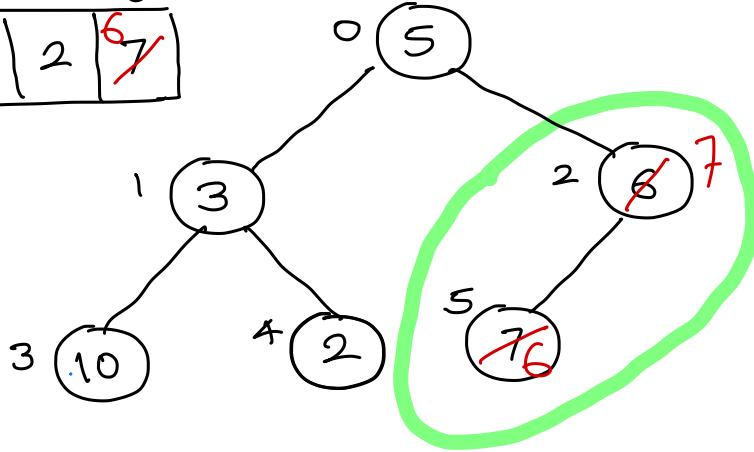
Last child node = $n-1$

Last parent node = $\left(\frac{n}{2} - 1\right)$

Build Max Heap

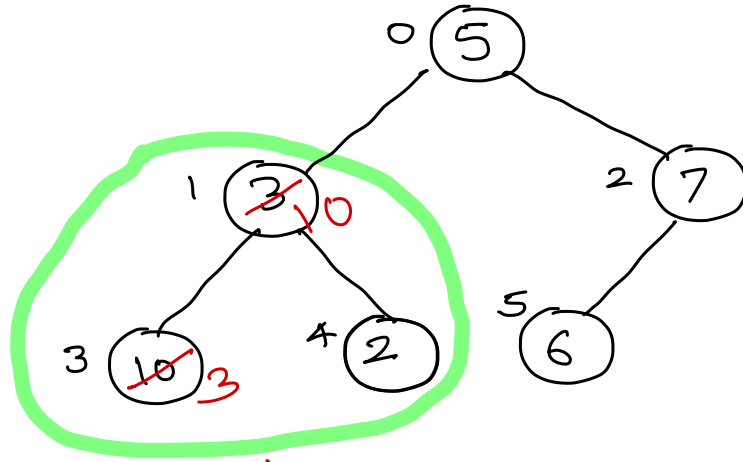
Start with last parent and move towards root.
Make sure each node satisfies heap property.

0	1	2	3	4	5
5	3	7 ⁶	10	2	6 ⁷



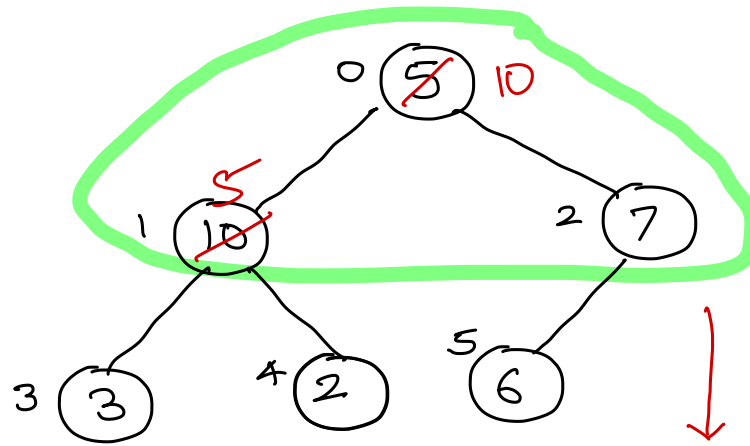
→ As this parent node do not satisfy heap property, swap parent's value with its child having largest value.

0	1	2	3	4	5
5	10 3	7	3 10	2	6



↓
 As this parent node do not satisfy heap property, swap parent's value with its child having largest value.

0	1	2	3	4	5
10 5	10 10	7	3	2	6

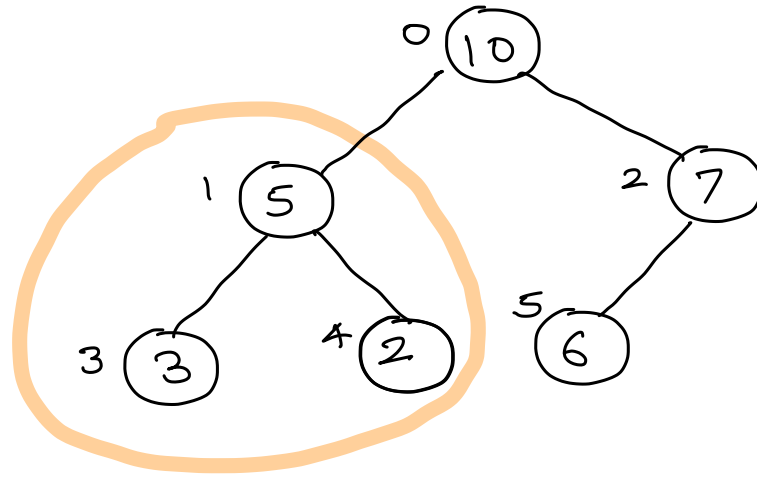


As this parent node do not satisfy heap property, swap parent's value with its child having largest value.

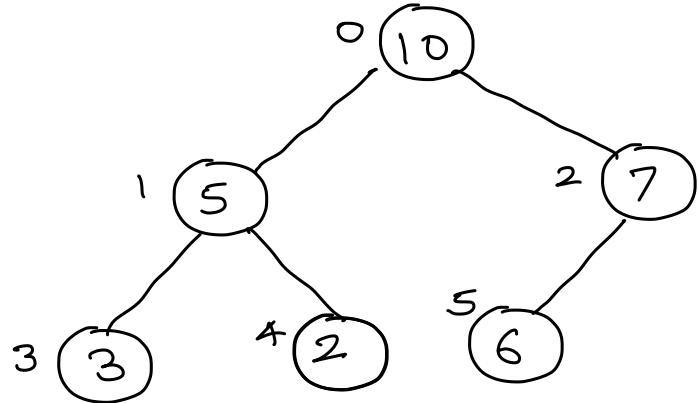
Since the updated child node is also a parent, we need to make sure it still satisfies heap property.

0	1	2	3	4	5
10	5	7	3	2	6

Satisfies heap property.



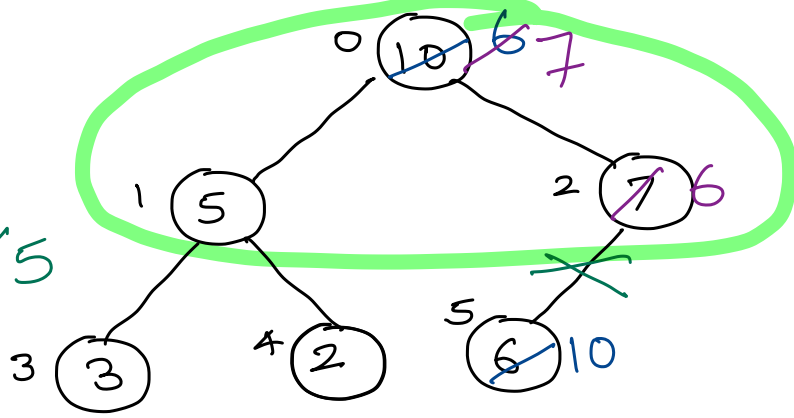
0	1	2	3	4	5
10	5	7	3	2	6



Max Heap

0	1	2	3	4	5
10 6	5	6 7	3	2	10 8

elements To Be Sorted = ~~6~~ 5

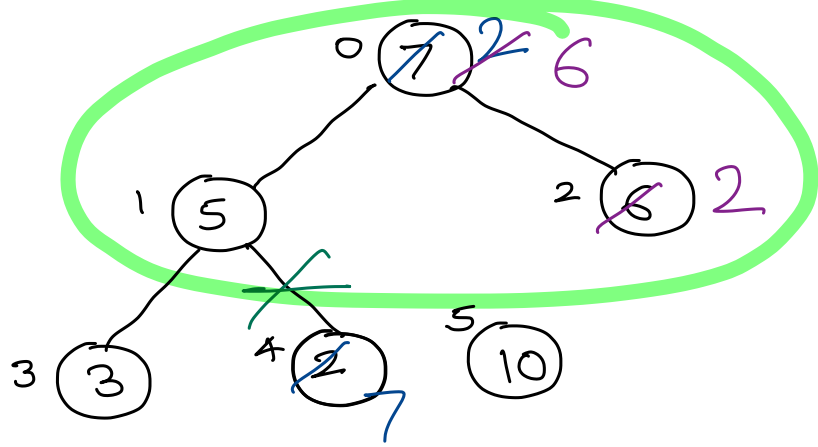


Once max heap is build, repeat the following steps while more than one element needs to be sorted.

- ① Swap value of root and last child.
- ② Remove last child from heap. (Reduce elements to be sorted by one.)
- ③ Make sure root satisfies heap property.

	0	1	2	3	4	5
6	7	5	8	3	2	10

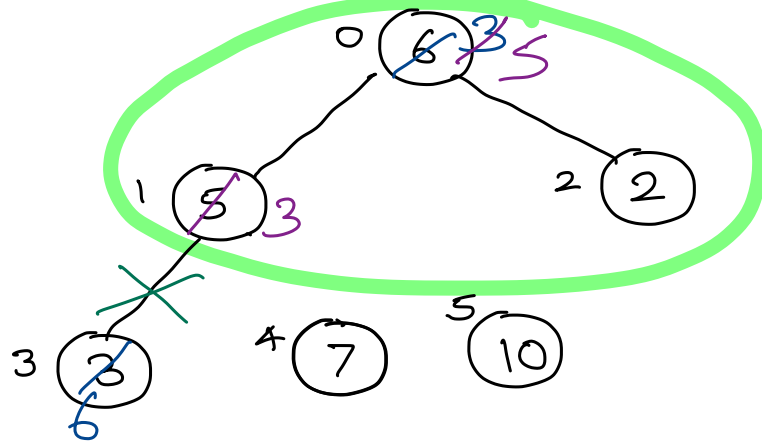
elements To Be Sorted = ~~5~~ 4



- ① Swap value of root and last child.
- ② Remove last child from heap. (Reduce elements to be sorted by one.)
- ③ Make sure root satisfies heap property.

0	1	2	3	4	5
8 6	3 5	2	6 3	7	10

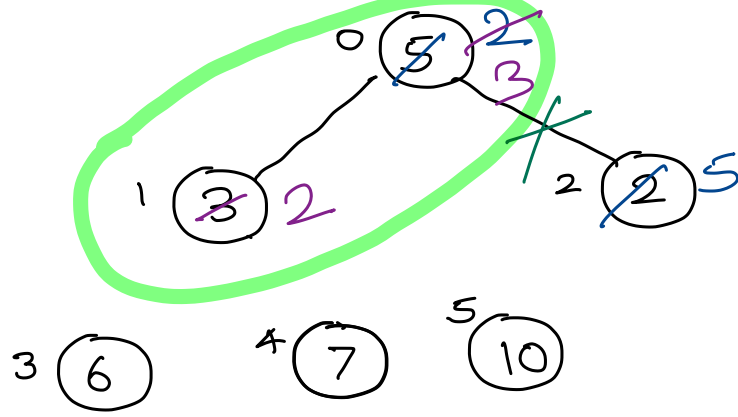
elements To Be Sorted = ~~4~~
3



- ① Swap value of root and last child.
- ② Remove last child from heap. (Reduce elements to be sorted by one.)
- ③ Make sure root satisfies heap property.

0	1	2	3	4	5
3 ² 5	3 ²	5 ² 2	6	7	10

Elements To Be Sorted = ~~3~~
2

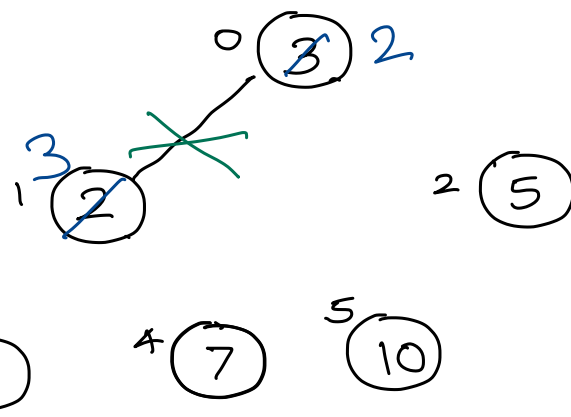


① Swap value of root and last child.

② Remove last child from heap. (Reduce elements to be sorted by one.)

③ Make sure root satisfies heap property.

0	1	2	3	4	5
2 3	3 2	5	6	7	10



Elements To Be Sorted = ~~2~~
1

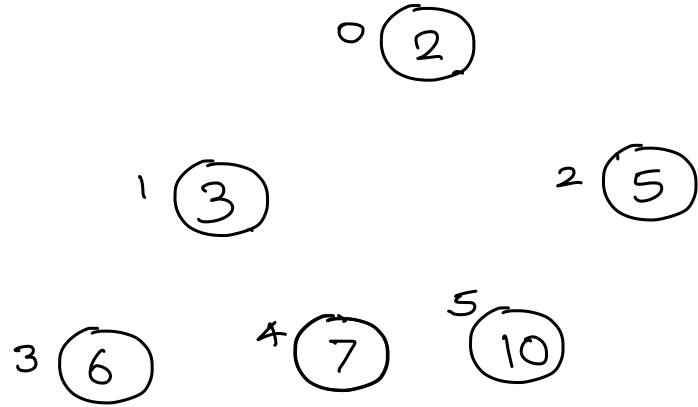
① Swap value of root and last child.

② Remove last child from heap. (Reduce elements to be sorted by one.)

③ Make sure root satisfies heap property.

0	1	2	3	4	5
2	3	5	6	7	10

elements To Be Sorted = 1



HeapSort(arr)

- ConvertToMaxHeap(arr, n)
- Set lastChildPos to $n - 1$
- while (lastChildPos > 0)
 - Swap root(0) and lastChildPos values
 - if (lastChildPos > 1)
 - MakeMaxHeap(arr, 0, lastChildPos)
 - Decrement lastChildPos by 1
- Stop

ConvertToMaxHeap(arr, n)

- Set lastParent to $n / 2 - 1$
- while (lastParent >= 0)
 - MakeMaxHeap(arr, lastParent, n)
 - Decrement lastParent by 1
- Stop

MakeMaxHeap(arr, parent, n)

// Find which child has largest value

- Set maxChildPos to $2 * \text{parent} + 1$

- Set rightChildPos to $2 * \text{parent} + 2$

// If right child exist, is it the one with value larger of two childs?

- if (rightChildPos < n)

- if (arr[rightChildPos] > arr[maxChildPos])

- maxChildPos = rightChildPos

// Check if parent has value larger than the largest child

- if (arr[parent] > arr[maxChildPos])

- Stop

// As child has larger value, swap it with parent

- Swap values at parent and maxChildPos

// Child value has changed, if its a parent node then it should still be a max heap

- if (maxChildPos is a leaf node) $\rightarrow (2 * \text{maxChildPos} + 1) \geq n$

- Stop

- MakeMaxHeap(arr, maxChildPos, n)

- Stop

↑
Tail Recursion

↑
left child